

SCJP 1.6 (CX-310-065 , CX-310-066)

Subject: Declarations and Access Control

Classes, interfaces, abstract, enum and var-args

Total Questions : 55

Prepared by : <http://www.javacertifications.net>

SCJP 6.0: Declarations, Initialization and Scoping
Questions Question - 1

What is the output for the below code ?

```
public class Test{
    int $7;
    public static void main(String argv[]){
        int k;
        Test test = new Test();
        test.$7=7;
        System.out.println(test.$7);
        System.out.println(k);
    }
}
```

1.7 0

2.0 0

3.Compile error - \$7 is not valid identifier.

4.Compile error - The local variable k may not have been initialized.

Explanation :

D is the correct answer.

The local variable k may not have been initialized. Local variable must be initialized before use. \$7 is valid identifier. Identifiers must start with a letter, a currency character (\$), or underscore (_). Identifiers cannot start with a number.

Question - 2

What is the output for the below code ?

```
public class Test{
    int $7;
    public static void main(String argv[]){
        int k;
        Test test = new Test();
        test.$7=7;
        k = k+9;
        System.out.println(test.$7);
        System.out.println(k);
    }
}
```

- 1.7 9
- 2.0 9
- 3.Compile error - \$7 is not valid identifier.
- 4.Compile error - The local variable k may not have been initialized.

Explanation :

D is the correct answer.

The local variable k may not have been initialized. Local variable must be initialized before use. \$7 is valid identifier. Identifiers must start with a letter, a currency character (\$), or underscore (_). Identifiers cannot start with a number.

Question - 3

What is the output for the below code ?

```
public class A {
    int k;
    boolean istrue;
    static int p;
    public void printValue() {
        System.out.print(k);
        System.out.print(istrue);
        System.out.print(p);
    }
}

public class Test{

    public static void main(String argv[]){

        A a = new A();

        a.printValue();
    }
}
```

- 1.0 false 0
- 2.0 true 0
- 3.0 0 0
- 4.Compile error - static variable must be initialized before use.

Explanation :

A is the correct answer.

Global and static variable need not be initialized before use. Default value of global and static int variable is zero. Default value of boolean variable is false. Remember local variable must be initialized before use.

Question - 4

What is the output for the below code ?

```
public class Test {  
    public static void main(String... args) {  
        int a =5 , b=6, c =7;  
        System.out.println("Value is "+ b +c);  
        System.out.println(a + b +c);  
        System.out.println("String "+(b+c));  
    }  
}
```

- 1.Value is 67 18 String 13
- 2.Value is 13 18 String 13
- 3.Value is 13 18 String
- 4.Compilation fails

Explanation :

A is the correct answer.

If the left hand operand is not a String then + operator treat as plus BUT if left hand operand is a String then + perform String concatenation.

Question - 5

What is the output for the below code ?

```
public class Test{  
    int _$;  
    int $7;  
    int 8h;  
    public static void main(String argv[]){  
  
        Test test = new Test();  
        test.$7=7;  
        test.8h=8;  
  
        System.out.print(test.$7);  
        System.out.print(test.8h);  
        System.out.print(test._$);  
    }  
}
```

- 1.7 8 0
- 2.7 0 0
- 3.Compile error - \$7 is not valid identifier.
- 4.Compile error - 8h is not valid identifier.

Explanation :

D is the correct answer.

\$7 is valid identifier. Identifiers must start with a letter, a currency character (\$), or underscore (_). Identifiers cannot start with a number. Therefore 8h is invalid identifier.

Question - 6

What is the output for the below code ?

```
public class Test{
    int _$;
    int $7;
    int do;
    public static void main(String argv[]){

        Test test = new Test();
        test.$7=7;
        test.do=9;

        System.out.println(test.$7);
        System.out.println(test.do);
        System.out.println(test._$);
    }
}
```

1.7 9 0

2.7 0 0

3.Compile error - \$7 is not valid identifier.

4.Compile error - do is not valid identifier.

Explanation :

D is the correct answer.

\$7 is valid identifier. Identifiers must start with a letter, a currency character (\$), or underscore (_). Identifiers cannot start with a number. You can't use a Java keyword as an identifier. do is a Java keyword.

Question - 7

What is the output for the below code ?

```
public abstract final class Test{
    static int _$;
    static int $7;
```

```

public static void main(String argv[]){

    System.out.println($7);
    System.out.println(_$);

}
}

```

1.0 0

2.Compile error - The class Test can be either abstract or final, not both

3.Compile error - \$7 is not valid identifier.

4.Compile error - _\$ is not valid identifier.

Explanation :

B is the correct answer.

Compile error - The class Test can be either abstract or final, not both. If you make a class abstract or final then you can't extend the class or can't instantiate the class. So Compiler complain. \$7, _\$ is valid identifier. Identifiers must start with a letter, a currency character (\$), or underscore (_). Identifiers cannot start with a number.

Question - 8

What is the output for the below code ?

```

public final class Test{
    static int _$;
    static int $7;

    public static void main(String argv[]){

        Test t = new Test();
        System.out.println($7);
        System.out.println(_$);

    }
}

```

1.0 0

2.Compile error - final class can't instantiated.

3.Compile error - \$7 is not valid identifier.

4.Compile error - _\$ is not valid identifier.

Explanation :

A is the correct answer.

Default value of global and static int variable is zero. Final class can be instantiated but not extended. \$7, _\$ is valid identifier. Identifiers must start with a letter, a currency character (\$), or underscore (_). Identifiers cannot start with a number.

Question - 9

What is the output for the below code ?

```
public abstract class Test{
    static int _$;
    static int $7;

    public static void main(String argv[]){

        Test t = new Test();
        System.out.println($7);
        System.out.println(_$);

    }
}
```

- 1.0 0
- 2.Compile error - abstract class can't instantiated.
- 3.Compile error - \$7 is not valid identifier.
- 4.Compile error - _\$ is not valid identifier.

Explanation :

B is the correct answer.

Compile error - abstract class can't instantiated. \$7, _\$ is valid identifier. Identifiers must start with a letter, a currency character (\$), or underscore (_). Identifiers cannot start with a number.

Question - 10

What is the output for the below code ?

```
private class Test{
    static int _$;
    static int $7;

    public static void main(String argv[]){

        Test t = new Test();
        System.out.println($7);
        System.out.println(_$);

    }
}
```

- 1.0 0
- 2.Compile error - outer class can't have private modifier.
- 3.Compile error - \$7 is not valid identifier.
- 4.Compile error - _\$ is not valid identifier.

Explanation :

B is the correct answer.

Compile error - outer class can't have private modifier. Only (public, abstract and final) are permitted. \$, _ \$ is valid identifier. Identifiers must start with a letter, a currency character (\$), or underscore (_). Identifiers cannot start with a number.

Question - 11

What is the output for the below code ?

```
public final class A {
    int k;
    boolean istrue;
    static int p;
    public void printValue() {
        System.out.print(k);
        System.out.print(istrue);
        System.out.print(p);
    }
}

public class B extends A{

    int k=5;
    boolean istrue;
    static int p=4;
    public void printValue() {
        System.out.print(k);
        System.out.print(istrue);
        System.out.print(p);
    }
}

public class Test{

    public static void main(String argv[]){

        A a = new B();
        a.printValue();
    }
}
```

1.5 false 4

2.Compile error - Can't subclass final classes.

3.0 false 0

4.5 true 4

Explanation :

B is the correct answer.

Compile error - final class can't be extended. Class A is final so can't extend. When final is used in a class declaration, the final keyword means the class can't be subclassed.

Question - 12

What is the output for the below code ?

```
public abstract class A {
    int k;
    boolean istrue;
    static int p;
    public void printValue() {
        System.out.print(k);
        System.out.print(istrue);
        System.out.print(p);
    }
}

public class B extends A{

    int k=5;
    boolean istrue;
    static int p=4;
    public void printValue() {
        System.out.print(k);
        System.out.print(istrue);
        System.out.print(p);
    }
}

public class Test{

    public static void main(String argv[]){

        A a = new B();
        a.printValue();
    }
}
```

1.5 false 4

2.Compile error - methods in abstract class can't have bodies.

3.0 false 0

4.5 true 4

Explanation :

A is the correct answer.

methods in abstract class can have bodies. But abstract class can't instantiated.

Question - 13

What is the output for the below code ?

```
public abstract class A {
    public abstract void printValue() {};
}

public class B extends A{

    int k=5;
    boolean istrue;
    static int p=4;
    public void printValue() {
        System.out.print(k);
        System.out.print(istrue);
        System.out.print(p);
    }
}

public class Test{

    public static void main(String argv[]){

        A a = new B();
        a.printValue();
    }
}
```

1.5 false 4

2.Compile error - Abstract methods do not specify a body.

3.0 false 0

4.5 true 4

Explanation :

B is the correct answer.

The methods marked abstract end in a semicolon rather than curly braces. If a method end with curly braces, it treats as body. Abstract methods do not specify a body.

Question - 14

What is the output for the below code ?

```
package com;
class Animal {

    public void printName(){
        System.out.println("Animal");
    }

}
```

```
package exam;
import com.Animal;
public class Cat extends Animal {

    public void printName(){
        System.out.println("Cat");
    }

}
```

```
package exam;
import com.Animal;

public class Test {

    public static void main(String[] args){
        Animal a = new Cat();
        a.printName();
    }

}
```

1. Animal
2. Cat
3. Animal Cat
4. Compile Error

Explanation :

D is the correct answer.

Cat class won't compile because its superclass, Animal, has default access and is in a different package. Only public superclass can be accessible for different package.

Question - 15

What is the output for the below code ?

```
public class A {
    int i = 10;
```

```

        public void printValue() {
            System.out.println("Value-A");
        };
    }

    public class B extends A{
        int i = 12;
        public void printValue() {
            System.out.print("Value-B");
        }
    }

    public class Test{

        public static void main(String argv[]){

            A a = new B();
            a.printValue();
            System.out.println(a.i);

        }
    }

```

- 1.Value-B 11
- 2.Value-B 10
- 3.Value-A 10
- 4.Value-A 11

Explanation :
B is the correct answer.

If you create object of subclass with reference of super class like (A a = new B();)
then subclass method and super class variable will be executed.

Question - 16

What is the output for the below code ?

```

public interface InfA {

    public final String getName();

}

public class A implements InfA{

    public final String getName() {
        return "Name";
    }

}

```

```

public class Test{

    public static void main(String argv[]){

        InfA a = new A();
        System.out.println(a.getName());

    }

}

```

- 1.Name
- 2.Compile error - Illegal modifier for the interface method InfA.getName()
- 3.Compile error - final method can't be implemented.
- 4.Compile successfully but Runtime Exception

Explanation :

B is the correct answer.

All interface methods are implicitly abstract., they cannot be marked final, strictfp, or native. Compiler complain - Illegal modifier for the interface method InfA.getName(); only public and abstract are permitted.

Question - 17

What is the output for the below code ?

```

public interface InfA {

    public static String getName();

}

public class A implements InfA{

    public static String getName() {
        return "Name";
    }

}

public class Test{

    public static void main(String argv[]){

        InfA a = new A();
        System.out.println(a.getName());

    }

}

```

- 1.Name
- 2.Compile error - Illegal modifier for the interface method InfA.getName()
- 3.Compile error - static method can't be implemented.
- 4.Compile successfully but Runtime Exception

Explanation :

B is the correct answer.

Interface methods must not be static. Compiler complain - Illegal modifier for the interface method InfA.getName(); only public and abstract are permitted.

Question - 18

What is the output for the below code ?

```
public interface InfA {  
    protected String getName();  
}  
  
public class A implements InfA{  
    public String getName() {  
        return "Name";  
    }  
}  
  
public class Test{  
    public static void main(String argv[]){  
        InfA a = new A();  
        System.out.println(a.getName());  
    }  
}
```

- 1.Name
- 2.Compile error - interface methods are always public.
- 3.Compile error - interface methods are always static.
- 4.Compile successfully but Runtime Exception.

Explanation :

B is the correct answer.

Interface methods are always public. You can't use protected or private. Compiler complain - Illegal modifier for the interface method InfA.getName(); only public and abstract are permitted.

Question - 19

What is the output for the below code ?

```
public interface InfA {
    int i = 10;
    public String getName();
}

public class A implements InfA{

    public String getName() {
        i = i +15;
        System.out.println(i);
        return "Name";
    }
}

public class Test{

    public static void main(String argv[]){

        InfA a = new A();
        System.out.println(a.getName());

    }
}
```

1.10 Name

2.Cimpile error - The final field InfA.i cannot be assigned

3.25 Name

4.Cimpile error - i is not accessible in class A

Explanation :

B is the correct answer.

All variables defined in an interface implicitly public, static, and final. Cimpile error - The final field InfA.i cannot be assigned. Because final variable cannot be modified.

Question - 20

What is the output for the below code ?

```

package exam;
public class A {

    protected int p = 10;

}

package other;
import exam.A;
public class B extends A{
    public void printValue() {
        System.out.print(p);
        A a = new A();
        System.out.println(a.p);
    }

}

```

```

package other;

public class Test {
    public static void main(String argv[]){
        B b = new B();
        b.printValue();
    }

}

```

- 1.10 10
- 2.Cimpile error - The field A.p is not visible
- 3.10 0
- 4.0 10

Explanation :
B is the correct answer.

Cimpile error - The field A.p is not visible. Because subclass outside the package, the protected member can be accessed only through inheritance. System.out.print(p) print 10 because protected member can be accessed only through inheritance if super class and sub class is in different packages. a.p is not visible in different package.

Question - 21

What is the output for the below code ?

```

package exam;
public class A {

    protected int p = 10;

}

```

```

package exam;

public class B extends A{

    public void printValue() {
        System.out.print(p);
        A a = new A();
        System.out.println(a.p);

    }
}

```

```

package exam;

public class Test {
    public static void main(String argv[]){
        B b = new B();
        b.printValue();
    }
}

```

- 1.10 10
- 2.Compile error - The field A.p is not visible
- 3.10 0
- 4.0 10

Explanation :
A is the correct answer.

protected member are accessible if they are in same package. In this case same package name "exam" so accessible for a.p . protected member can be accessed only through inheritance if super class and sub class is in different packages.

Question - 22

What is the output for the below code ?

```

package exam;
public class A {

    int p = 10;

}

package other;
import exam.A;
public class B extends A{
    public void printValue() {
        System.out.print(p);
    }
}

```



```

    }

}

package other;
public class Test {
    public static void main(String argv[]){
        B b = new B();
        b.printValue();
    }
}

```

- 1.10
- 2.Compile error - The field A.p is not visible
- 3.0
- 4.Compile clean but Runtime Exception

Explanation :
B is the correct answer.

default members are visible to subclasses only if those subclasses are in the same package as the superclass.

Question - 23

What is the output for the below code ?

```

public class Test{

    public static void main(String argv[]){

        Test t = new Test();
        t.printValue();

    }

    public void printValue(){
        private int k = 10;
        System.out.println(k);
    }
}

```

- 1.10
- 2.Compile error - local variable declared with an access modifier will not compile.
- 3.0
- 4.Compile clean but Runtime Exception

Explanation :
B is the correct answer.

Any local variable declared with an access modifier will not compile. private int k = 10; will not compile because access modifier private is used. Only final can be permitted.

Question - 24

What is the output for the below code ?

```
public class Test{

    public static void main(String argv[]){

        Test t = new Test();
        t.printValue();

    }

    public void printValue(){
        private int k = 10;
        System.out.println(k);
    }
}
```

- 1.10
- 2.Compile error - local variable declared with an access modifier will not compile.
- 3.0
- 4.Compile clean but Runtime Exception

Explanation :

B is the correct answer.

Any local variable declared with an access modifier will not compile. private int k = 10; will not compile because access modifier private is used. Only final can be permitted.

Question - 25

What is the output for the below code ?

```
public enum Test {
    BREAKFAST(7, 30), LUNCH(12, 15), DINNER(19, 45);

    private int hh;

    private int mm;

    Test(int hh, int mm) {
        assert (hh >= 0 && hh <= 23) : "Illegal hour.";
        assert (mm >= 0 && mm <= 59) : "Illegal mins.";
        this.hh = hh;
        this.mm = mm;
    }
}
```

```

    }

    public int getHour() {
        return hh;
    }

    public int getMins() {
        return mm;
    }

    public static void main(String args[]){
        Test t = new BREAKFAST;
        System.out.println(t.getHour() + ":" + t.getMins());
    }
}

```

1.7:30

2.Compile Error - an enum cannot be instantiated using the new operator.

3.12:30

4.19:45

Explanation :

B is the correct answer.

As an enum cannot be instantiated using the new operator, the constructors cannot be called explicitly. You have to do like Test t = BREAKFAST;

Question - 26

What is the output for the below code ?

```

public class Test {
    enum Day {
        MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY,
        SUNDAY
    }
    enum Month {
        JAN, FEB
    }

    public static void main(String[] args) {

        int[] freqArray = { 12, 34, 56, 23, 5, 13, 78 };

        // Create a Map of frequencies
        Map ordinaryMap = new HashMap();
        for (Day day : Day.values()) {
            ordinaryMap.put(day, freqArray[day.ordinal()]);
        }

        // Create an EnumMap of frequencies
        EnumMap frequencyEnumMap = new EnumMap(ordinaryMap);

        // Change some frequencies
        frequencyEnumMap.put(null, 100);
    }
}

```

```

        System.out.println("Frequency EnumMap: " + frequencyEnumMap);
    }
}

```

- 1.Frequency EnumMap: {MONDAY=12, TUESDAY=34, WEDNESDAY=56, THURSDAY=23, FRIDAY=5, SATURDAY=13, SUNDAY=78}
- 2.Compile Error
- 3.NullPointerException
- 4.Frequency EnumMap: {MONDAY=100, TUESDAY=34, WEDNESDAY=56, THURSDAY=23, FRIDAY=5, SATURDAY=13, SUNDAY=123}

Explanation :

C is the correct answer.

The null reference as a key is NOT permitted.

Question - 27

```

public class EnumTypeDeclarations {

    public void foo() {

        enum SimpleMeal {
            BREAKFAST, LUNCH, DINNER
        }

    }

}

```

Is the above code Compile without error ?

- 1.Compile without error
- 2.Compile with error
- 3.Compile without error but Runtime Exception
- 4.Compile without error but Enum Exception

Explanation :

B is the correct answer.

An enum declaration is a special kind of class declaration: a) It can be declared at the top-level and as static enum declaration. b) It is implicitly static, i.e. no outer object is associated with an enum constant. c) It is implicitly final unless it contains constant-specific class bodies, but it can implement interfaces. d) It cannot be declared abstract unless each abstract method is overridden in the constant-specific class body of every enum constant. e) Local (inner) enum declaration is NOT OK! Here in public void foo() { enum SimpleMeal { BREAKFAST, LUNCH, DINNER } } enum declaration is local within method so compile time error.

Question - 28

What is the output for the below code ?

```
public enum Test {
    int t;
    BREAKFAST(7, 30), LUNCH(12, 15), DINNER(19, 45);

    private int hh;

    private int mm;

    Test(int hh, int mm) {
        assert (hh >= 0 && hh <= 23) : "Illegal hour.";
        assert (mm >= 0 && mm <= 59) : "Illegal mins.";
        this.hh = hh;
        this.mm = mm;
    }

    public int getHour() {
        return hh;
    }

    public int getMins() {
        return mm;
    }

    public static void main(String args[]){
        Test t = BREAKFAST;
        System.out.println(t.getHour() + ":" + t.getMins());
    }
}
```

- 1. 7:30
- 2. Compile with error
- 3. 12:15
- 4. 19:45

Explanation :

B is the correct answer.

The enum constants must be declared before any other declarations in an enum type. In this case compile error because of declaration int t; before enum declaration.

Question - 29

What is the output for the below code ?

```
1. public class A {
2.     static int add(int i, int j){
3.         return i+j;
4.     }
5. }
```

```

6. public class B extends A{
7.     public static void main(String argv[]){
8.         short s = 9;
9.         System.out.println(add(s,6));
10.    }
11.}

```

1. Compile fail due to error on line no 2
2. Compile fail due to error on line no 9
3. Compile fail due to error on line no 8
4. 15

Explanation :

D is the correct answer.

The Short s is autoboxed correctly and output is 15.

Question - 30

What is the output for the below code ?

```

1. public class A {
2.     int add(int i, int j){
3.         return i+j;
4.     }
5. }
6. public class B extends A{
7.     public static void main(String argv[]){
8.         short s = 9;
9.         System.out.println(add(s,6));
10.    }
11.}

```

1. Compile fail due to error on line no 2
2. Compile fail due to error on line no 9
3. Compile fail due to error on line no 8
4. 15

Explanation :

B is the correct answer.

Cannot make a static reference to the non-static method add(int, int) from the type A.
The short s is autoboxed correctly, but the add() method cannot be invoked from a static method because add() method is not static.

Question - 31

What is the output for the below code ?

```

1. public class B {

```

```

2. public static void main(String argv[]){
3.     short s = 9;
4.     System.out.println(add(s,6));
5. }
6. public int add(int i, int j){
7.     return i+j;
8. }
9. }

```

- 1.Compile fail due to error on line no 3
- 2.Compile fail due to error on line no 4
- 3.Compile fail due to error on line no 7
- 4.15

Explanation :

B is the correct answer.

Cannot make a static reference to the non-static method add(int, int) from the type B.
The short s is autoboxed correctly, but the add() method cannot be invoked from a static method because add() method is not static.

Question - 32

What is the output for the below code ?

```

1. public class EnumTest {
2.     public enum Months { JAN, FEB, MAR, APR};
3.     public static void main(String argv[]){
4.         Months [] mon = Months.values();
5.         System.out.println(mon[3]);
6.     }
7. }

```

- 1.Compile fail due to error on line no 4
- 2.Compile fail due to error on line no 5
- 3.MAR
- 4.APR

Explanation :

D is the correct answer.

enum type has a static values method that returns an array containing all of the values of the enum type in the order they are declared.

Question - 33

Which of the following statement is true?

- 1.outer class can only declare public , abstract and final

- 2.outer class may be private
- 3.outer class can't be abstract
- 4.outer class can't be final

Explanation :

A is the correct answer.

outer class can only declare public , abstract and final

Question - 34

Is the bellow statement is true?

"abstract class cannot be instantiated, but can be extended by subclasses"

- 1.true
- 2.false
- 3.partially true
- 4.None of the above

Explanation :

A is the correct answer.

abstract class cannot be instantiated, but can be extended by subclasses.

Question - 35

What is the output for the below code ?

```
public interface TestInf {  
    int i =10;  
}  
  
public class Test {  
    public static void main(String... args) {  
        TestInf.i=12;  
        System.out.println(TestInf.i);  
    }  
}
```

- 1.Compile with error
- 2.10
- 3.12
- 4.0

Explanation :

A is the correct answer.

All the variables declared in interface is Implicitly static and final , so can't change the value.

Question - 36

What is the output for the below code ?

```
public class Test {  
    public static void main(String... args) {  
        final int i;  
        System.out.println(i);  
    }  
}
```

- 1.Compile with error
- 2.0
- 3.Runtime Exception
- 4.None of the above

Explanation :

A is the correct answer.

final variables should be initialized.

Question - 37

What will be the result of compiling the following code:

```
public class Test {  
  
    public static void main(String... args) {  
        static int i;  
        System.out.println(i);  
    }  
}
```

- 1.the code will compile and print 0
- 2.the code will compile an print null
- 3.Compile with error.
- 4.Compile clean but Runtime Exception

Explanation :

C is the correct answer.

static variables should not be local.

Question - 38

What is the output for the below code ?

```
public class A extends Integer{  
    public static void main(String[] args){  
        System.out.println("Hello");  
    }  
}
```

- 1.Hello
- 2.Compile Error
- 3.Compile clean but Runtime Exception
- 4.null

Explanation :

B is the correct answer.

final class can't be extended. Integer is final class.

Question - 39

```
1. public class Point {  
2.     int x = 1, y = 1;  
3.     abstract void alert();  
4. }
```

Is the above code compile without error ?

- 1.compile without error
- 2.Compilation fails due to an error on line 3
- 3.Compilation fails due to an error on line 1
- 4.Compilation fails due to an error on line 2

Explanation :

B is the correct answer.

If there is any abstract method in a class then the class should be abstract. The abstract method alert in type Point can only be defined by an abstract class.

Question - 40

Which of the following statement is true?

- 1.A subclass of an abstract class that is not itself abstract may be instantiated.

- 2.abstract class can't be instantiated.
- 3.final class can't be extended.
- 4.All of the above

Explanation :

D is the correct answer.

All statements are true.

Question - 41

What is the correct ordering for the import, class and package declarations when found in a Java class?

- 1.package, import, class
- 2.class, import, package
- 3.import, package, class
- 4.package, class, import

Explanation :

A is the correct answer.

Example : package com; import java.io.IOException; public class Test { }

Question - 42

What will be the result of compiling the following code:

```
public class Test {  
    public static void main (String args []) {  
        int age;  
        age = age + 1;  
        System.out.println("The age is " + age);  
    }  
}
```

- 1.the code will compile an print 1.
- 2.the code will compile an print 0.
- 3.Compile time error.
- 4.None of the above

Explanation :

C is the correct answer.

local variable should be initialized.

Question - 43

What will be the result of compiling the following code:

```
public class Test {  
    static int age;  
    public static void main (String args []) {  
        age = age + 1;  
        System.out.println("The age is " + age);  
    }  
}
```

- 1.The code does not compile.
- 2.The code compiles cleanly and print 1.
- 3.The code compiles cleanly and print 0.
- 4.The code throws an Exception at Runtime.

Explanation :

B is the correct answer.

static variables are initialized automatically. default initialization value for int is 0 and boolean is false.

Question - 44

Which of the following can be used to define a constructor for this class, given the following code:

```
public class Test {  
    ...  
}
```

- 1.public void Test() {...}
- 2.public Test() {...}
- 3.public static Test() {...}
- 4.public static void Test() {...}

Explanation :

B is the correct answer.

Constructor should not have any return type and should not be static.

Question - 45

What is the output for the below code ?

```
1. public class Test {  
2.     static { int a = 5; }
```

```

3.          public static void main(String[] args) {
4.          System.out.println(a);
5.          }
6.          }

```

- 1.Compilation fails due to an error on line 2
- 2.Compilation fails due to an error on line 4
- 3.Compilation fails due to an error on line 1
- 4.5

Explanation :

B is the correct answer.

A variable declared in a static initialiser is not accessible outside its enclosing block.

Question - 46

What is the output for the below code ?

```

class c2
{
    {
        System.out.println("initializer");
    }

    public static void main(String a[])
    {
        System.out.println("main");
        c2 ob1=new c2();
    }
}

```

- 1.prints main and initializer
- 2.prints initializer and main
- 3.Compilation fails
- 4.Compilation succeed but Runtime Exception

Explanation :

A is the correct answer.

statement block executes on creation of object of the class.

Question - 47

What is the output for the below code ?

```

public class A {
    static{System.out.println("static");}
    { System.out.println("block");}
}

```

```

    public A(){
        System.out.println("A");
    }

    public static void main(String[] args){
        A a = new A();
    }

```

- 1.A block static
- 2.static block A
- 3.static A
- 4.A

Explanation :

B is the correct answer.

First execute static block, then statement block then constructor.

Question - 48

What is the output for the below code ?

```

class A {
    { System.out.print("b1 "); }
    public A() { System.out.print("b2 "); }
}
class B extends A {
    static { System.out.print("r1 "); }
    public B() { System.out.print("r2 "); }
    { System.out.print("r3 "); }
    static { System.out.print("r4 "); }
}
class C extends B {
    public static void main(String[] args) {
        System.out.print("pre ");
        new C();
        System.out.println("post ");
    }
}

```

- 1.r1 r4 pre b1 b2 r3 r2 post
- 2.r1 r4 pre b1 b2 post
- 3.r1 r4 pre b1 b2 post r3 r2
- 4.pre r1 r4 b1 b2 r2 r3 post

Explanation :

A is the correct answer.

All static blocks execute first then blocks and constructor. Blocks and constructor executes (super class block then super class constructor, sub class block then sub class constructor). Sequence for static blocks is super class first then sub class. Sequence for blocks is super class first then sub class.

Question - 49

What is the output for the below code ?

```
class Bird {  
    int i;  
    boolean b;  
    float f;  
  
    public Bird() {  
        System.out.println(i);  
        System.out.println(b);  
        System.out.println(f);  
    }  
}
```

- 1.0 false 0.0
- 2.0 true 0.0
- 3.0 null 0
- 4.null 0 0

Explanation :

A is the correct answer.

JVM initialize default values; FOR boolean default value is false.

Question - 50

What is the output for the below code ?

```
1. public class Test{  
2.     public static void main(String[] args){  
3.         System.out.println("The answer is: "+18+3);  
4.     }  
5. }
```

- 1.The answer is: 21
- 2.The answer is: 183
- 3.Compilation fails due to an error on line 3
- 4.The answer is: null

Explanation :

B is the correct answer.

Concatenated as String so print "The answer is: 183".

Question - 51

What is the output for the below code ?

```
class C {
public static void main(String[] args) {
    int i1=1;
    switch(i1){
        case 1:
            System.out.println("one");

        case 2:
            System.out.println("two");

        case 3:
            System.out.println("three");
    }
}}
```

1. prints one two three
2. prints one
3. compile error
4. Runtime exception

Explanation :

A is the correct answer.

There is no break statement in case 1 so it causes the below case statements to execute regardless of their values

Question - 52

What is the output for the below code ?

```
1. class C1
2. {
3.     static class C2
4.     {
5.         static int i1;
6.     }
7. public static void main(String a[])
8. {
9.     System.out.println(C1.C2.i1);
10. }
11. }
```

1. prints 0
2. Compilation fails due to an error on line 9
3. Compilation fails due to an error on line 3
4. Compilation fails due to an error on line 1

Explanation :

A is the correct answer.

static members can be accessed without instantiating the particular class

Question - 53

What is the output for the below code ?

```
1. public class Test{
2.     public static void main(String[] args){
3.         int[] a1[] = new int[3][3];
4.         int a2[4] = {3,4,5,6};
5.         int a2[5];
6.     }
7. }
```

1. Compilation fails due to an error on lines 3,4,5
2. Compilation fails due to an error on lines 4,5
3. Compilation fails due to an error on line 3
4. Compilation fails due to an error on line 1

Explanation :

B is the correct answer.

no value should be specified in the right side brackets when constructing an array

Question - 54

What is the output for the below code ?

```
public class Test{
    int i;
    public static void main (String[] args) {
        int i; //1
        private int a = 1; //2
        protected int b = 1; //3
        public int c = 1; //4
        System.out.println(a+b+c); //5
    }
}
```

1. compile time error at lines 1,2,3,4,5
2. compile time error at lines 2,3,4,5
3. compile time error at lines 2,3,4
4. prints 3

Explanation :

C is the correct answer.

The access modifiers public, protected and private, can not be applied to variables declared inside methods.

Question - 55

What is the output for the below code ?

```
public class Test {
```

```
public static void main(String... args) throws Exception {  
    Integer i = 34;  
    int l = 34;  
    if(i.equals(l)){  
        System.out.println(true);  
    }else{  
        System.out.println(false);  
    }  
}  
}
```

- 1.true
- 2.false
- 3.Compilation fails
- 4.Runtime Exception

Explanation :

A is the correct answer.

equals() method for the integer wrappers will only return true if the two primitive types and the two values are equal.